Further K215 proof simplification: one source language and smaller proof boundaries

Research report prepared for Vladimir Reshetnikov

22 October 2026

Abstract

The recent commits have implemented several earlier simplifications. The next useful step is not another broad abstraction: it is to remove redundant representations while retaining the existing proof boundaries. We give an exact source-language identification for the separately checked decoder-prefix component, a small proposal to infer AX macro words from their actual programs, and a Rocq proof that can reuse its existing loop theorem. We distinguish changes to the maintained endpoint from preparation for a future decoder integration. All new formal files are uncompiled drafts; finite data audits are reproducible and do not replace kernel checks.

1 Current baseline and recommended priorities

This review is pinned to ProveIt commit `cbf94cb552a3b49509e0eee80e7455d4eb5974c6`. The target remains the accepted 215-state, two-symbol, blank-input machine with dense-table SHA-256

`fdbcef0825886035c3091c8537a69535bc96f9e265f0ae45863543d40ae70090`.

No machine transition, source program, calculus or consistency statement is changed. The existing endpoint relates blank halting to a finite ZFC refutation. Its negation gives the desired nonhalting/consistency equivalence. This review proposes refactoring the external Lean and Rocq proofs; it does not provide a new derivation encoded in ZFC's own proof syntax.

The following are no longer new opportunities at this pin:

- The shared implication core is in the maintained Rocq source; the old large `CD222` and `DomRho8` developments have short adapters.
- Lean constructs the full seed on the original model. The preliminary indiscernibility quotient is gone. The later bisimulation quotient remains essential.
- The QHyp quotient is shared; source trees are computed from programs; the Rocq source certificate is normalized and one-sided.
- Rocq's unused zero-seed enumerator layer is removed. Lean's AX words no longer repeat successor-node annotations.

The repository's archived 89-node macro-transport prototypes are also real prior work, not a new discovery. Their maintained-source integration is still unfinished.

Recommendation. For an immediate, low-risk production edit, try the Rocq loop reuse in Section 4. For a small representation cleanup, try the computed AX words in Section 3. For the next substantial decoder-proof simplification, remove the second source language as in Section 2, before further enlarging the prefix/suffix adapter. That last change simplifies a separately checked component and its future integration; it does *not* immediately shrink today's endpoint closure.

2 Delete the decoder's second source language

2.1 An exact identification, not an analogy

The decoder-prefix development has its own 202-node `sourceNodes` representation. The maintained register program `p215` already represents precisely the same source with 179 fork nodes and 23 distinct primitive edges. The difference is that the decoder assigns additional node numbers to primitive leaves.

Let ϕ map the j-th fork, in existing decoder-node order, to the register-program edge node(j), and each primitive node to its literal `RegisterPath.Edge`. Then for every fork v and branch b,

$$\phi(\text{child}(v, b)) = \text{branch}(p215, j, b), \qquad \phi(201) = \text{node}(178).$$

All 179 local squares are checked against the pinned definitions. This is not a minimization of the Turing machine or a claim that it has 179 states.

The complete role retargeting preserves the genuinely physical data:

Quantity	Unchanged count
Physical decoder roles	234
Arbitrary-tape blocks	281
Role branches	468
Internal / terminal branches	371 / 97
Expanded source leaves	4751
Counted guarded site outcomes	6176

A role is not identified by its source node alone. Its physical state, short suffix, role index and block references must survive. In particular, merging 234 roles into 179 source labels would be unsound without a separate argument.

2.2 The correspondence detour that can disappear

The current prefix route starts from `RegisterPath.walk`, passes through the tree `ptree p215` and a `LeafAt` witness, identifies that tree with the 203-fuel unfolding of the separate decoder syntax, and reconstructs a second `SourcePath`. Only then does it build a route through the checked physical blocks.

Instead, let roles point directly to nodes of `p215`. An internal role branch checks that the selected program edge is node(c), the target role has node c, and $c < n$. It retains the suffix and block-window conditions exactly. A terminal role branch uses the selected program edge itself, not a fictitious primitive-node number. This removes the intermediate syntax, its unfolding fuel, its tree-equality binding and the source-path/leaf correspondence from the intended architecture.

This is stronger than replacing a literal tree by a computed tree. It is also different from choosing a route before quantifying over tapes: the latter fixes the quantifier order but can still carry two source languages. The two improvements are compatible.

2.3 The new theorem that is actually needed

The finite label map alone is not the proof. The remaining generic theorem should say:

> For every admissible decoder role and every successful walk of the actual program from its source node, the role certificate supplies a finite block route to the corresponding primitive-entry port. The route preserves the consumed depth, unread suffix, physical state and local window. Its clock and port are chosen before the arbitrary ambient tape.

Prove this by induction on the successful walk, or on the decreasing source-node number for internal branches. At each internal step, use the existing arbitrary-tape block theorem, update the suffix to the last two bits of the known local word, and invoke the induction hypothesis. At a terminal step, return the selected literal primitive edge and the actual physical action.

The critical obligations are:

1. Node bounds and strict decrease for every internal target. The 179 finite squares and 468 branch checks establish the required literal facts, not universal execution semantics.
2. Correct accounting for the walk's zero extension and consumed depth. Trailing zero erasure and arbitrary unread bits cannot be silently identified with a different physical entry convention.
3. Nonempty block windows and positive block durations, using the retained block checks. A source-rank decrease does not itself prove physical execution.
4. A terminal port indexed by the *actual source edge*, together with the original physical state/action and suffix.
5. The existing residual primitive and return-to-dispatch contracts, and the existing arbitrary-tape-to-zipper reflection.

A useful counterexample to an overclaim. The pilot's terminal checker tests that the source edge is non-internal and that the physical transition equals the stored action. Changing one primitive opcode to another still non-internal opcode does not change that test. Therefore it cannot prove increment/decrement/break/continue semantics. This is intentional for a prefix checker, but it makes the residual contracts mandatory. The audit's 6,176 outcomes are counted and structurally mapped, not newly executed or kernel-proved outcomes.

2.4 Migration boundary

First split the formal transition table and 281 block certificates out of `K215Decoder`, so that the direct-program checker can import them without importing the old source language. Then replace the source-index fields and prove the walk-to-route theorem. Keep old names as compatibility wrappers until their clients have been checked.

Five prefix modules are involved, from the certificate definition to `KDecoderPrefix215`; the companion proposal gives their exact interfaces and the dependency split. Today's `KSim`, `KSites215` and its 105 site blocks remain unchanged at this stage. Only a later semantic-contract integration can remove work from the production endpoint. The supplied checker pilot deliberately still imports the old block provider, so it is not itself a dependency reduction. Net source and build savings must be measured after that split and integration.

3 Compute AX words, but retain their checker

The recent successor-inference change leaves one 156-atom list in each of `L2AxiomsPerm` and `L2AxiomsDom`. Every atom is already determined by the left macro edge of its literal program node. There are only thirteen macro kinds.

Define a partial reverse lookup from `AXAtom.node`. A small generic extractor follows the right successor, returning the list of decoded atoms; it stops at gate 457 and uses fuel 156 from entry 236. Define `axWord` to be its result, with the empty list as the failure default. Crucially, leave the existing checker and theorem in place:

$$\mathrm{axWordCheck}(236, \mathrm{axWord}) = \mathrm{true}.$$

The existing `axWordRun` applies to *every* accepted word. Therefore no new theorem asserting correctness of the extractor is needed. A wrong reverse lookup or insufficient fuel must still pass the old checker; the empty default fails at entry 236 because $236 \neq 457$. The extractor is a producer, not a new soundness assumption.

Both literal words are recovered exactly by the finite audit. The two words differ only in positions 7 and 8: `readp1, cons` versus `cons, readp1`. The proposal does not identify these words or assume that their actions commute. Each is checked against its own literal DAG.

The supplied minimal overlays change only the import, the thirteen-atom reverse lookup and the definition of `axWord`. The text of `axWordCheck`, `axWordRun`, `axWord_checked`, `axWordClean` and `ax_eval` is unchanged. The maintained `generate_l2axiomsdom.py` template must also be updated; the supplied overlays are not a complete producer patch. Those proofs nevertheless must be elaborated again. In particular, the existing `simp only` proof of `ax_eval` may need reduction support now that the word is computed rather than literal.

Honest size accounting. The two source modules total 13,039 bytes. The overlays total 11,520 bytes and the new helper is 732 bytes: a net reduction of 787 bytes. Line count instead rises from 310 to 316. For the K215 lane alone, counting the entire new helper, the saving is only 27 bytes and one module is added. The benefit is removing redundant macro annotations and their maintenance, not a substantial endpoint-size reduction. Kernel reduction may cost more, as both extraction and checking traverse the program. No timing improvement is claimed. If literal words are faster enough to justify their storage, keep the current representation.

The same pattern may be useful in Rocq, but this packet does not claim a Rocq AX-word implementation. It is safer to pilot the existing Lean checker boundary than to introduce a second generic semantic interpreter merely for symmetry between languages.

4 Rocq: reuse the existing loop proof

`ProgLibPerm.z465` still proves a register-clearing loop by induction on its initial value. The existing `sem_while` and `while_tac` already cover that exact shape. Nodes 199 and 465 have the same rows:

$$(\text{EDecnz } 5,\ \text{EContinue } 1).$$

Keep the old public statement, including its now-unused premise:

$$\forall a\ m,\quad \text{get}(m, 5) = a \implies \text{sem}(\text{ENode } 465, m, \text{XNormal}, \text{set}(m, 5, 0)).$$

After introducing m and discarding the premise, apply `while_tac` with identity body transformer and post-state $m \mapsto \text{set}(m, 5, 0)$.

The obligations are the literal row, $5 \leq 8$, the existing continuation rule at level one, preservation of the tested register by the identity body, setting an already-zero register, and overwrite of the same register. No functional extensionality, new termination measure or additional semantic hypothesis is needed. Applying `w199` directly at node 465 would be wrong; the proposal instead reuses the generic theorem already used to prove `w199`.

The declaration changes from 9 to 5 lines and from 475 to 171 bytes. The two calls in `r466` keep their types. Apply the edit to the maintained `NEW_EQ` producer template as well as the generated file, then regenerate and compile the downstream endpoint. The patch and typed comparison client are supplied but uncompiled.

The larger Rocq opportunity is integration, not rediscovery. The maintained Perm and 219 macro bodies remain a 753-line renamed duplicate. Archived `common-macros` and `collection-first` prototypes already implement the 89-node transport idea. Their reported Coq 8.18 component checks are not a fresh Rocq 9.1 endpoint check. Production integration still needs a shared state-record identity, exact public wrappers, producer synchronization, and preservation of every exit and final register value. This remains higher-impact than the local four-line cleanup, but should be credited as adoption of existing work.

5 A deliberately deferred abstraction

Two maintained Lean proofs, `KLowerRel.sat_relT` and `WFRelativization.sat_translate`, share the mathematics of interpreting formulas in a definable subtype. We developed a capture-aware common kernel as an experiment, but do *not* recommend adopting it merely to reduce current source size.

The correct abstraction needs a binding context c, its update c^+, a slot map σ_c, an ambient invariant $G(c, r)$ and a guard formula g_c. The laws are

$$\begin{aligned} &\sigma_{c^+}(0) = 0, && \sigma_{c^+}(i+1) = \sigma_c(i) + 1, \\ &G(c,r) \Rightarrow G(c^+, x :: r), && \mathrm{Sat}(x :: r, g_c) \longleftrightarrow P(x) \quad \text{under } G(c,r). \end{aligned}$$

If $r(\sigma_c(i)) = (e(i)).\mathrm{val}$, induction on the formula gives

$$\mathrm{Sat}_V(r, \mathrm{translate}_c(\varphi)) \longleftrightarrow \mathrm{Sat}_{\{x|P(x)\}}(e, \varphi).$$

The guarded universal and existential cases are the substantive ones. The theorem itself does not need choice or a global inhabitedness assumption.

For the tagged domain, the context is the moving marker index d, the guard is membership from slot $d+1$ to slot zero after pushing a variable, and $G(d, r)$ says $r(d) = t$. For the well-founded subtype, the context is trivial, slots are unchanged and the guard is the existing well-foundedness formula at zero. The named-variable constructibility translation uses a different syntax and is *not* included in the claimed reuse.

The draft kernel is 108 lines and its comparison client is 71 lines, before production integration and compatibility work. The two existing translators and satisfaction proofs total 89 lines, or 99 including the tagged environment helpers. Factoring two inductions into a richer interface is not automatically a shortening. Exact old-translation identities, equality-free clients and binder/capture obligations still need checking. The packet retains this experiment and finite capture regressions for future reuse, but does not count it as a proven size saving. A smaller reuse of existing subtype quantifier lemmas may be preferable unless more clients justify the full interface.

6 Validation and adoption plan

6.1 What was checked here

The archive contains pinned source snapshots, explicit mappings, proposed overlays, finite audit programs and an independent data checker. The checks read source as inert data. They do not execute repository generators or invoke Lean, Rocq or a kernel checker.

The finite audits establish the exact node/role correspondence and both inferred AX words. They verify the unchanged Rocq statement and loop row, and independently reproduce the source-size comparisons. Negative controls include malformed or non-internal rows, insufficient fuel, incorrect atom decoding and the wrong continuation level. The subtype checks exercise capture and equality cases on finite structures; they are regression tests, not a quantified proof.

No new formal file in this packet has been compiled. No endpoint dependency count, proof-term size or build-time saving is claimed. Historical full builds and recent per-module checks remain repository evidence; this review does not upgrade those records into a fresh full build at the review pin.

6.2 Acceptance gates

1. Apply a single proposal at a time in an isolated worktree, retaining the pinned machine tables and endpoint theorem statements.

2. For Rocq, synchronize the shared producer template, regenerate every affected output, compile the changed library and callers, and check the exact sparse and dense endpoints and their assumptions. The proposed proof must be closed under the same global context.

3. For computed AX words, compile the helper, both edited modules, their root and seed consumers, and the affected endpoints. Check assumptions, exact regeneration, and both compile-time and source-size effects. Do not infer success from unchanged theorem text.

4. For the direct-program decoder, first check the finite retargeting and retained block certificates, then prove the universal walk-to-route theorem and its arbitrary-tape reflection. Only after residual primitive contracts are composed should the production site interface be changed.

5. Preserve compatibility adapters until all intended clients are checked. Measure maintained sources and generators together, rather than counting only deleted generated copies.

Bottom line. The most promising architectural simplification is to make the literal program the decoder's sole source syntax. The safest immediate Rocq simplification is reuse of an existing loop theorem. The computed AX-word pilot tests the same principle on a small production component without adding a new soundness theorem. Broad semantic abstraction should follow demonstrated reuse, not be treated as a saving by itself.

Pinned primary sources

- K215 verification baseline and endpoint boundaries.
- Prior proposal catalogue, distinguishing archived prototypes from maintained integration.
- Lean AX checker and word, and Rocq macro library.

The accompanying provenance files give exact URLs and hashes for every source used in the component audits.